

- In real life When making crucial decisions, everyone consider the help of numerous specialists rather than relying on the advice of a single person.
- Ensemble learning is the same way of working. Ensemble learning is the process of combining numerous individual model to produce a better model.
- The Ensemble model has better accuracy and reliability.

Ensemble Learning is a method to predict by fusing the salient properties of two or more models.

Type of Ensemble Learning: -

1. Bagging technique
2. Boosting technique
3. Voting technique

1. Bagging

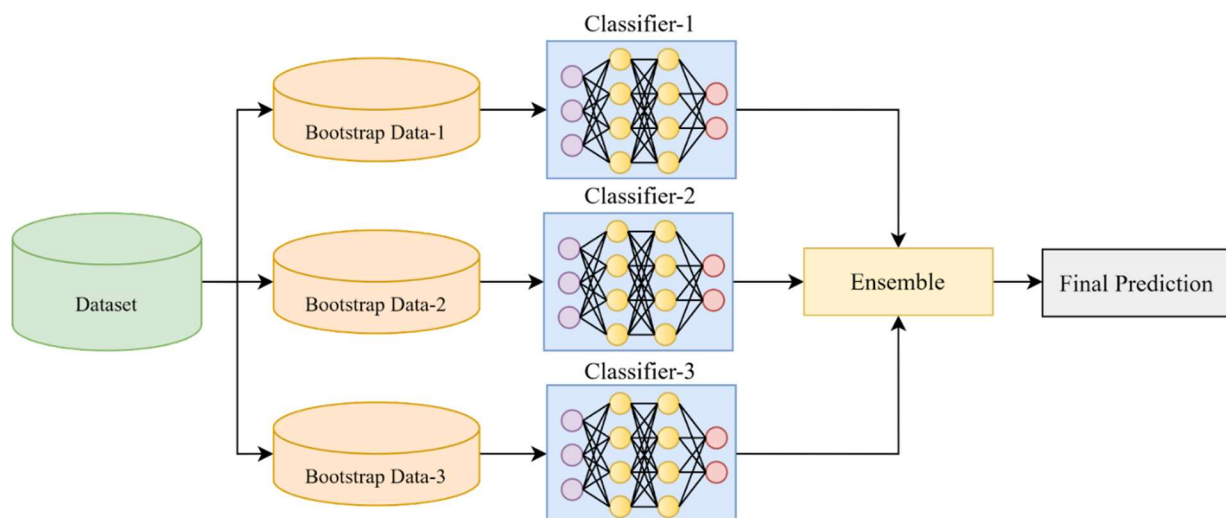
The Bagging ensemble technique (“bootstrap aggregating”)

Random subsets of a dataset are **created using replacement**, (meaning that the same data point may be present in several subsets).

These subsets are called “bootstrap sampling.”

These subsets are now treated as independent datasets, on which Machine Learning algorithms applied.

There is an aggregation mechanism used to compute the final prediction (like averaging, weighted averaging, etc.).



Bagging ensemble mechanism.

Parallel stream of processing occurs. **The main aim of the bagging method is to reduce variance in the ensemble predictions.**

Popular ensemble methods:

- Bagged Decision Trees
- **Random Forest Classifiers**
- Extra Trees

Bagging vs Random forest: -

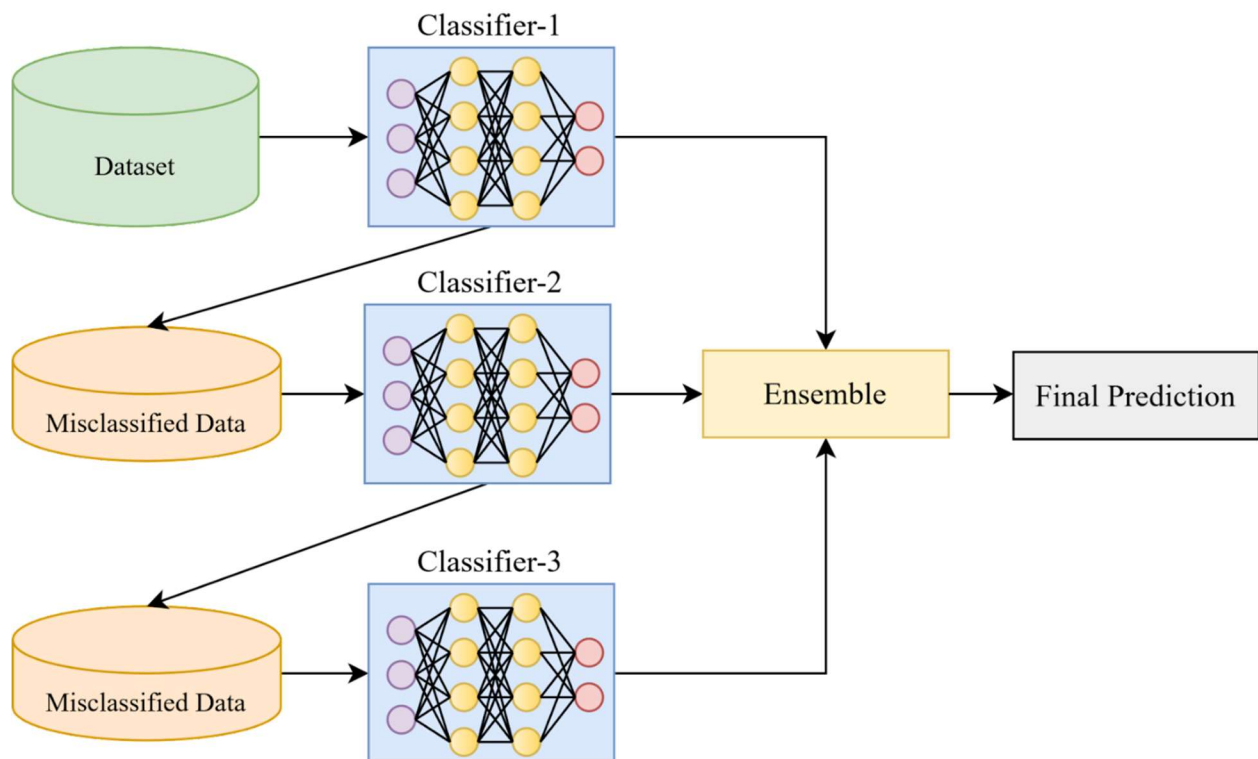
OOB (out of bag data):-

2. Boosting

The boosting ensemble mechanism works different from the bagging mechanism.

Here, instead of parallel processing of data, sequential processing of the dataset occurs.

The first classifier is fed with the entire dataset, and the predictions are analyzed.



The instances where Classifier-1 fails to produce correct predictions (that are samples near the decision boundary of the feature space) are fed to the second classifier.

This is done so that Classifier-2 can specifically focus on the problematic areas of feature space and learn an appropriate decision boundary.

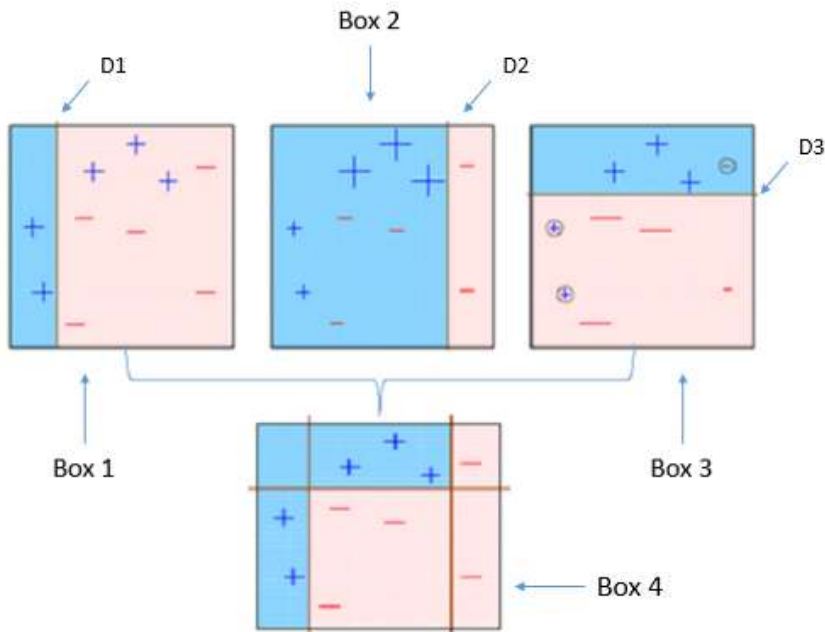
Similarly, further steps of the same idea are employed, and then the ensemble of all these previous classifiers is computed to make the final prediction on the test data.

Boosting ensemble mechanism

The main aim of the boosting method is to reduce bias in the ensemble decision. Other algorithms based on this approach include:

- Adaptive Boosting
- Stochastic Gradient Boosting or Gradient Boosting

Adaptive Boosting Machines

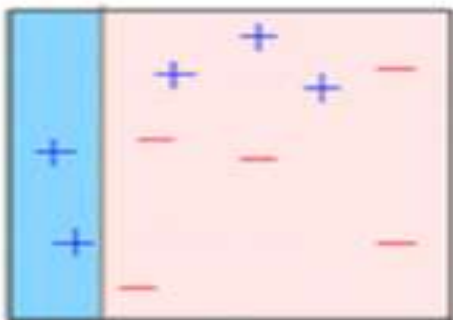


This diagram aptly explains Ada-boost.

Iteration 1.

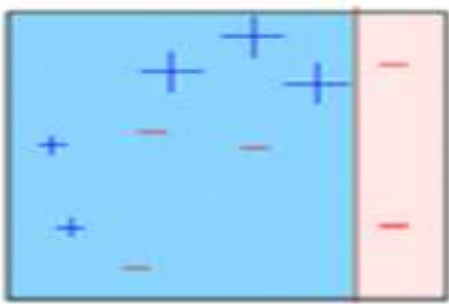
Box 1: assigned equal weights to each data point and applied a decision stump to classify them as + (plus) or – (minus). The decision stump (D1) has generated vertical line at left side to classify the data points.

This vertical line has incorrectly predicted three + (plus) as – (minus). In such case, we'll assign higher weights to these three + (plus) and apply another decision stump.



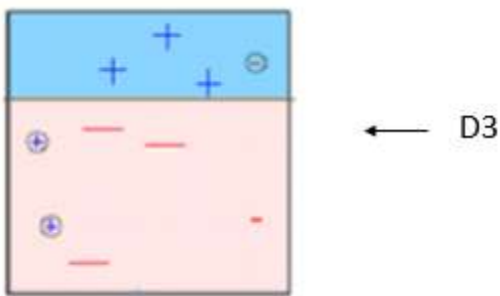
Iteration 2.

Box 2: Here, we can see that the size of three incorrectly predicted + (plus) is bigger as compared to rest of the data points. In this case, the second decision stump (D2) will try to predict them correctly. Now, a vertical line (D2) at right side of this box has classified three mis-classified + (plus) correctly. But again, it has caused mis-classification errors. This time with three -(minus). Again, we will assign higher weight to three – (minus) and apply another decision stump.



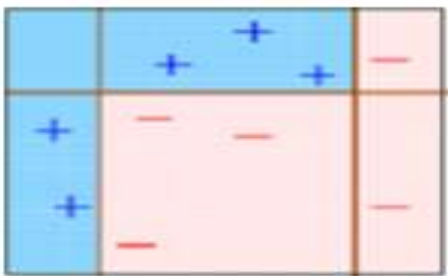
Iteration 3

Box 3: Here, three – (minus) are given higher weights. A decision stump (D3) is applied to predict these mis-classified observation correctly. This time a horizontal line is generated to classify + (plus) and – (minus) based on higher weight of mis-classified observation.



Iteration 4

Box 4: Here, we have combined D1, D2 and D3 to form a strong prediction having complex rule as compared to individual weak learner. You can see that this algorithm has classified these observation quite well as compared to any of individual weak learner.



Important :-AdaBoost (Adaptive Boosting) It starts by predicting original data set and gives equal weight to each observation. If prediction is incorrect using the first learner, then it gives higher weight to observation which have been predicted incorrectly. Being an iterative process, it continues to add learner(s) until a limit is reached in the number of models or accuracy.

Stochastic Gradient Boosting or Gradient Boosting

In gradient boosting, it trains many model sequentially. Each new model gradually minimizes the loss function ($y = ax + b + e$, e needs special attention as it is an error term) of the whole system using **Gradient Descent** method. The learning procedure consecutively fit new models to provide a more accurate estimate of the response variable.

The principle idea behind this algorithm is to construct new base learners which can be maximally correlated with negative gradient of the loss function, associated with the whole ensemble.

3. Majority Voting

Majority voting is one of the earliest and easiest ensemble schemes

In this method, an odd number of contributing classifiers are chosen, and for each sample, the predictions from the classifiers are computed. the class that gets most of the class from the classifier pool is deemed the ensemble's predicted class.

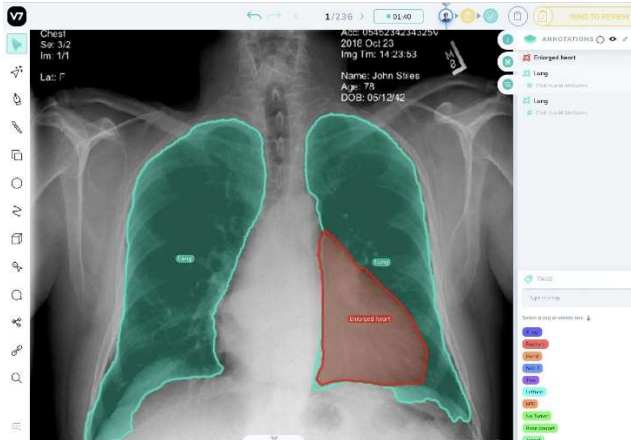
Such a method works well for binary classification problems, where there are only two candidates for which the classifiers can vote.

However, it fails for a problem with many classes since many cases arise, where no class gets a clear majority of the votes.

Applications of Ensemble Learning

1. Disease detection

cardiovascular disease detection from X-Ray and CT scans.



2. Remote Sensing

Tasks like Landslide Detection and Scene Classification



3. Fraud Detection

4. Speech emotion recognition

1. What is the difference between hard and soft voting classifiers?

Answer: A hard voting classifier just counts the votes of each classifier in the ensemble and outputs the class which was in majority. On the other hand, a soft voting classifier calculates the class probability for each classifier in the ensemble and outputs the class with the highest probability. But soft voting only works for the classifier which has the `predict_proba()` method.

2. Do you think you can speed up the training time of bagging by distributing it across multiple servers? If yes, then why? What about Pasting, boosting, and Random Forest?

Answer: Yes it is possible to speed up the training time of bagging by distributing them on multiple servers because all of them are independent of each other. Pasting and Random Forest also works similarly. So no issue with them too. Talking about boosting algorithms they rely on their predecessors. So it is not possible to train them across multiple servers.

3. What are the advantages of Out-Of-Bag Evaluation?

Answer: As we all know that in the out-of-bag evaluation, each predictor is evaluated on the instances on which it was not trained (37% that were left, if you remember). This gives an advantage of having an unbiased evaluation of the ensemble without any need for a validation set.

4. How are Extra Trees any different from a Regular Random Forest? Between Extra Trees and random Forest which is faster? How can Randomness help in extra trees?

Answer: Both Extra Trees and Random Forest considers a random subset of features for splitting at each node. The only difference is that instead of searching for the best possible threshold for a feature, extra trees use a random threshold. This randomness of features acts as regularization, if ever Random Forest is overfitting the training data there is a good chance that Extra trees might perform better. In terms of prediction speed, both of them are the same, as we know that Extra Trees don't look for the best threshold for a feature, it saves it a lot of time. Hence, making it faster than Random Forest.

5. While working you find that the AdaBoost classifier is under fitting the training data, what are the hyper parameters you will tweak, and how?

Answer: If we observe that AdaBoost is underfitting the training data, we can try on increasing the number of estimators or we can also try reducing the regularization hyperparameters of the base estimator. Increasing the learning rate might also help a bit.

6. While working you find that your Gradient Boosting is overfitting the training data, what are your thoughts about the learning rate?

Answer: If we observe that Gradient Boosting is overfitting the training data, we can go ahead and reduce the learning rate, and also we can use early stopping (another hot topic will discuss some other day) to find the right number of predictors.

Automatic Hyper Parameter Tuning

What is a hyper parameter?

- algorithms are math their success depends on what hyper parameters were used .
- Hyper parameters values like k in KNN, kernel in SVM, *alpha* in Regularization regression (Ridge Lasso), $n_estimator$ in tree based algorithms etc..
- They strictly control the fit of the model, for each dataset, there is a unique set of optimal hyper parameters to be found.
- The most basic way of finding this perfect set would be randomly trying out different values based on gut feeling.
- However, this manual method becomes useless when there are many hyper parameters to tune.

Most common methods for automatic hyper parameter tuning: **Random search** and **Grid search**.

Given a set of possible values for all hyper parameters of a model,

- A Grid search fits a model using every single combination of Given set of all hyper parameters
- In each fit, the Grid search uses cross-validation to account for overfitting. After all combinations are tried, the search retains the parameters that resulted in the best score so that you can use them to build your final model.

Random search takes a bit different approach than Grid.

- Instead of exhaustively trying out every single combination of hyper parameters, which can be computationally expensive and time-consuming, it randomly samples hyper parameters and tries to get closer to the best set.

Scikit-learn

provides `GridSearchCV` and `RandomizedSearchCV` classes that make this process!

Randomized Search with Sklearn `RandomizedSearchCV`

Scikit-learn

provides `RandomizedSearchCV` class to implement random search.

It requires two arguments to set up: **an estimator** and the set of possible values for hyperparameters called a ***parameter grid*** or *space*.

This parameter grid dictionary should have hyper parameters as keys in the syntax they appear in the model's documentation. The possible **parameter grid** values can be given as an array.

The main difference between these two techniques is the **obligation to try all parameters**.

GridSearchCV has to try ALL the parameter combinations, however, RandomSearchCV can choose only a few 'random' combinations out of all the available combinations.

Drawback - GridSearchCV will go through all the intermediate combinations of hyper parameters which makes grid search computationally very expensive.

RandomizedSearchCV:

RandomizedSearchCV solves the drawbacks of GridSearchCV, as it goes through only a fixed number of hyperparameter settings.

It moves within the grid in a random fashion to find the best set hyper parameters. This approach reduces unnecessary computation.